

Master Notes — Linked Lists & Trees

Part 1a: Singly / Doubly / Circular Linked Lists, Sentinel Nodes (C++,
FAANG-condensed)

Contents

1	Singly Linked List	2
2	Doubly Linked List	9
3	Circular Linked List	12
4	Sentinel / Dummy / Header Nodes	14

1 Singly Linked List

One-line definition

A linear structure of nodes where each node holds data and a pointer to the next node; traversal is one-directional.

Intuition

```
head
|
v
[10|*]-->[20|*]-->[30|*]-->NULL
```

You only ever know "what's next," never "what's before." Every SLL algorithm carries just enough extra state (**prev** pointer, **slow/fast** pair) to compensate for the missing backward edge.

Graph Intuition

- A directed graph, out-degree ≤ 1 per node, acyclic in the well-formed case.
- **Reverse** is $O(n)/O(1)$ **space** because each **next** pointer is rewritten exactly once — a bijection between old and new edges.
- **Floyd's cycle detection works** because once both pointers are inside a cycle of length c , the fast pointer closes the gap by 1 node/step, guaranteeing a meeting within c steps (two runners on a circular track).

Memory Layout

- Heap-allocated, non-contiguous $\Rightarrow$ poor cache locality (every dereference is a likely cache miss).
- **head** is a stack variable (pointer) unless it's a class member.
- Ownership: prefer `unique_ptr<Node>` for automatic cleanup, or manual `delete` to avoid leaks.

Operations

1. Insertion (**head** / **tail** / **after node**)

Complexity: head $O(1)$; tail $O(n)$ without tail pointer, $O(1)$ with one; after-node $O(1)$.

Common bug: losing the old head reference by updating **head** before wiring `node->next`.

```
struct Node { int val; Node* next; Node(int v): val(v), next(nullptr) {}
};

Node* insertHead(Node* head, int val) {
    Node* node = new Node(val);
    node->next = head;
    return node; // new head
}
```

2. Deletion (by value / given only a node pointer)

Complexity: $O(n)$ to find; $O(1)$ if pointer + predecessor already held.

Trap (classic FAANG question): "delete a node given only a pointer to it (not head)" — copy the *next* node's value into *cur*, then delete the next node instead. **Fails if cur is the tail** (nothing to fake being the last node) — always mention this limitation.

```
Node* deleteVal(Node* head, int val) {
    Node dummy(0); dummy.next = head;
    Node *prev = &dummy, *cur = head;
    while (cur) {
        if (cur->val == val) { prev->next = cur->next; delete cur; break; }
        prev = cur; cur = cur->next;
    }
    return dummy.next;
}

void deleteGivenNode(Node* node) {           // node guaranteed not tail
    Node* nxt = node->next;
    node->val = nxt->val;
    node->next = nxt->next;
    delete nxt;
}
```

3. Search

$O(n)$, $O(1)$ space. No binary search — no random access.

4. Reverse (iterative, preferred) & Recursive

Idea: three-pointer walk *prev*, *cur*, *next*; snapshot *cur->next* before overwriting.

Invariant: after k nodes processed, $[0..k)$ is fully reversed; *cur* is the first untouched node.

Iterative: $O(n)$ time, $O(1)$ space. **Recursive:** $O(n)$ time, $O(n)$ stack — risk of stack overflow for $n > 10^5$; always mention the iterative version is safer.

```
Node* reverse(Node* head) {
    Node *prev = nullptr, *cur = head;
    while (cur) {
        Node* nxt = cur->next;
        cur->next = prev;
        prev = cur; cur = nxt;
    }
    return prev;
}

Node* reverseRec(Node* head) {
    if (!head || !head->next) return head;
    Node* newHead = reverseRec(head->next);
    head->next->next = head;
    head->next = nullptr;
    return newHead;
}
```

5. Middle Node (slow/fast)

Fast moves 2 steps, slow moves 1; when fast ends, slow is at the middle. $O(n)$ time, $O(1)$ space, single pass.

Even length: loop condition `fast && fast->next` returns the *second* middle; use `fast->next && fast->next->next` to get the first.

```
Node* middle(Node* head) {
    Node *slow = head, *fast = head;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    return slow;
}
```

6. Cycle Detection & Cycle Start (Floyd's Tortoise and Hare)

Why it detects a cycle: gap shrinks by 1/step once both are inside; meets within c steps, else fast hits nullptr.

Why the "reset to head" trick finds the start: let d = distance head $\rightarrow$ entry, x = distance entry $\rightarrow$ meeting point, c = cycle length. Slow travels $d + x$; fast travels $2(d + x) = d + x + nc$, so $d = nc - x$. Walking d steps from the meeting point (mod c) lands back at the entry, and walking d steps from head also lands at the entry — so both pointers, advanced 1 step at a time, meet exactly at the cycle start.

```
bool hasCycle(Node* head) {
    Node *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next; fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}

Node* cycleStart(Node* head) {
    Node *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next; fast = fast->next->next;
        if (slow == fast) {
            Node* p = head;
            while (p != slow) { p = p->next; slow = slow->next; }
            return p;
        }
    }
    return nullptr;
}
```

7. Merge Two Sorted Lists & Merge Sort

Merge: dummy node + pick smaller head each step. $O(n + m)$ time, $O(1)$ extra space.

Merge sort: split via slow/fast, recursively sort halves, merge. $O(n \log n)$ time, $O(\log n)$ stack.

Preferred over quicksort on lists because the merge step needs only sequential access ($O(1)$ extra space), whereas quicksort partitioning wants efficient random-access swaps that lists don't give.

```

Node* mergeTwo(Node* a, Node* b) {
    Node dummy(0); Node* tail = &dummy;
    while (a && b) {
        if (a->val <= b->val) { tail->next = a; a = a->next; }
        else { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;
    return dummy.next;
}

Node* sortList(Node* head) {
    if (!head || !head->next) return head;
    Node *slow = head, *fast = head->next;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->
        next; }
    Node* mid = slow->next; slow->next = nullptr;
    return mergeTwo(sortList(head), sortList(mid));
}

```

8. Remove N-th Node From End

Two pointers separated by n ; when the leading one hits null, the trailing one sits before the target. $O(L)$ time, one pass, $O(1)$ space. Dummy node lets "delete head" reuse the same code path.

```

Node* removeNthFromEnd(Node* head, int n) {
    Node dummy(0); dummy.next = head;
    Node *fast = &dummy, *slow = &dummy;
    for (int i = 0; i < n; i++) fast = fast->next;
    while (fast->next) { fast = fast->next; slow = slow->next; }
    Node* toDelete = slow->next;
    slow->next = slow->next->next;
    delete toDelete;
    return dummy.next;
}

```

9. Palindrome Check

Find middle $\rightarrow$ reverse second half $\rightarrow$ compare $\rightarrow$ (optionally restore). $O(n)$ time, $O(1)$ space (vs. $O(n)$ with an array copy). **Always mention restoring the list afterward** — interviewers expect it.

10. Intersection of Two Lists

Two pointers, each switches to the other list's head after exhausting its own; both traverse $len(A) + len(B)$ steps total, so they meet at the intersection or both hit null together.

```

Node* getIntersection(Node* a, Node* b) {
    Node *p1 = a, *p2 = b;
    while (p1 != p2) { p1 = p1 ? p1->next : b; p2 = p2 ? p2->next : a; }
    return p1;
}

```

11. Reverse in K-Groups, Rotate, Partition (idea + complexity only)

- **K-Groups:** reverse first k nodes, recurse/iterate on remainder, stitch. $O(n)$ time, $O(1)$ – $O(n/k)$ space. **Trap:** verify $\geq k$ nodes remain before reversing, else leave the final partial group untouched.
- **Rotate List:** compute length, make circular, break at $\text{len} - k\% \text{len}$ from head. $O(n)/O(1)$.
- **Partition around x :** build separate less-than and $\geq$ chains, splice. $O(n)/O(1)$, stable (preserves relative order).

12. Clone List with Random Pointer

$O(n)$ space: hashmap old $\rightarrow$ new.

$O(1)$ extra space (classic trick): interleave clones ($A \rightarrow A' \rightarrow B \rightarrow B' \rightarrow \dots$), set $A'.\text{random} = A.\text{random}.\text{next}$, then de-interleave.

```
struct RNode { int val; RNode *next, *random; };
RNode* copyRandomList(RNode* head) {
    if (!head) return nullptr;
    for (RNode* cur = head; cur; ) {
        RNode* clone = new RNode{cur->val, cur->next, nullptr};
        RNode* nxt = cur->next; cur->next = clone; cur = nxt;
    }
    for (RNode* cur = head; cur; cur = cur->next->next)
        if (cur->random) cur->next->random = cur->random->next;
    RNode dummy{0, nullptr, nullptr}; RNode* tail = &dummy;
    for (RNode* cur = head; cur; cur = cur->next) {
        tail->next = cur->next; tail = tail->next;
        cur->next = cur->next->next;
    }
    return dummy.next;
}
```

Patterns

Interviewer says	Think
"find middle / detect cycle"	slow/fast pointers
"kth from end / rotate"	two pointers with fixed gap k
"reverse (sub)list"	three-pointer iterative reversal
"merge k sorted lists"	divide and conquer or min-heap
"clone list with extra pointer"	interleaving or hashmap
"delete duplicates from sorted"	single pass, compare with next

Recognition: no random-access requirement, emphasis on $O(1)$ extra space, phrasing like "in-place," "without extra array," "single pass."

Complexity Table

Operation	Time	Space	Recursion depth
Search	$O(n)$	$O(1)$	–
Insert head	$O(1)$	$O(1)$	–
Insert tail (no tail ptr)	$O(n)$	$O(1)$	–
Delete	$O(n)$	$O(1)$	–
Reverse (iter)	$O(n)$	$O(1)$	0
Reverse (rec)	$O(n)$	$O(n)$	$O(n)$
Cycle detect	$O(n)$	$O(1)$	–
Merge sort	$O(n \log n)$	$O(\log n)$	$O(\log n)$

Interview Follow-ups — Answered

- **”Can you do it the other way (recursive/iterative)?”** Yes for reverse and most traversal problems — iterative trades cleaner code for saved stack space; state the $O(n)$ vs $O(1)$ space tradeoff explicitly.
- **”Can you do it in $O(1)$ space?”** Usually yes if you’re allowed to mutate the list in place (reversal-based palindrome check, interleaving trick for random-pointer clone) — the tradeoff is you must restore or destroy the original structure.
- **”What if the list is circular?”** Detect first with Floyd’s algorithm; if circular, cap traversal by node count or by ”stop when you return to start,” never by null-check.
- **”What if you can’t allocate new nodes?”** Reuse existing nodes via pointer rewiring (reverse, partition, merge) instead of copying values into new nodes — nearly every classic problem above is solvable this way.
- **”Restore the list to its original state afterward”** — required for palindrome check and any ”non-destructive” variant: re-reverse the second half / de-interleave clones before returning.

Competitive Programming Notes

- Prefer arrays / `next[]` index arrays (”static linked list”) over pointer nodes for speed and cache locality.
- Josephus-style: simulate with a circular list for small n ; use $J(n, k) = (J(n-1, k) + k) \bmod n$, $J(1, k) = 0$ for large n .

Advanced Topics

- **XOR Linked List:** `XORptr = prev XOR next` gives bidirectional traversal with single-pointer memory footprint; requires remembering the previous address while walking (no random jumps), and is essentially unusable with ASLR/modern GC languages.
- **Unrolled Linked List:** each node holds a small array/block of elements — fewer pointer-chases, better cache locality, more complex split/merge on insert/delete. Used in text-editor buffers.
- **Persistent / Functional Lists:** immutable nodes; ”modifying” creates new nodes sharing the unchanged suffix ($O(1)$ prepend, structural sharing) — this is exactly Lisp `cons`/Haskell lists.

- **Intrusive Lists:** `next/prev` fields live inside the object itself (Linux kernel `struct list_head`) — avoids extra allocation, lets one object belong to multiple lists.
- **Lock-free Linked Lists:** atomic CAS on pointers + Harris's marked pointers (low bit marks logical deletion) for concurrent insert/delete without a global lock; hazard pointers / epoch-based reclamation solve ABA / use-after-free.
- **Skip List:** probabilistic multi-level linked list, random "height" per node, $O(\log n)$ expected search/insert/delete with no rebalancing (Redis sorted sets, LevelDB).

Common Bugs

- Null-pointer dereference: forgetting `fast && fast->next` before `fast->next->next`.
- Losing the rest of the list by overwriting `next` before saving it.
- Off-by-one in "kth from end" gap sizing.
- Infinite loop from an accidentally-created cycle during relinking.

Checklist

1. Empty / single-node handled explicitly?
2. Do I need a dummy node?
3. Slow/fast, or fixed-gap two-pointer?
4. Saving `next` before overwriting it?
5. Need to restore the list afterward?
6. Is $O(1)$ extra space required?

Practice Problems

Difficulty	Problem	Source
Easy	Reverse Linked List	LeetCode 206
Easy	Middle of the Linked List	LeetCode 876
Easy	Linked List Cycle	LeetCode 141
Medium	Linked List Cycle II	LeetCode 142
Medium	Remove Nth Node From End	LeetCode 19
Medium	Reorder List	LeetCode 143
Medium	Add Two Numbers	LeetCode 2
Medium	Copy List with Random Pointer	LeetCode 138
Medium	Sort List	LeetCode 148
Hard	Reverse Nodes in k-Group	LeetCode 25
Hard	Merge k Sorted Lists	LeetCode 23
Very Hard	LFU Cache (list + hashmap design)	LeetCode 460
CSES	Josephus Problem I/II	CSES

2 Doubly Linked List

One-line definition

Each node stores pointers to both next and previous, giving $O(1)$ bidirectional traversal and $O(1)$ deletion given only a node pointer.

Intuition

```
NULL <- [10] <-> [20] <-> [30] -> NULL
      head           tail
```

The extra `prev` pointer removes the single biggest SLL weakness: true $O(1)$ deletion without needing a predecessor, plus backward traversal.

Graph Intuition

Treat `next/prev` as two consistent directed edge sets that must stay in sync (`a.next=b` $\iff$ `b.prev=a`). Every correctness proof reduces to "both directions of every edge updated together."

Memory Layout

Each node carries 2 pointers instead of 1 (more overhead, still non-contiguous, double the pointer-chase potential). This is the structure behind `std::list` and `java.util.LinkedList`.

Operations

1. Insertion / Deletion (given a node)

$O(1)$ given the node reference — the entire point of DLL over SLL. **Common bug:** rewiring in the wrong order and losing a reference — always save neighbor pointers into locals first.

```
struct DNode { int val; DNode *prev, *next; };

void insertAfter(DNode* node, int val) {
    DNode* n = new DNode{val, node, node->next};
    if (node->next) node->next->prev = n;
    node->next = n;
}

void removeNode(DNode* node) {
    if (node->prev) node->prev->next = node->next;
    if (node->next) node->next->prev = node->prev;
    delete node;
}
```

2. Traversal

$O(n)$ either direction — bidirectionality is exactly what powers LRU cache design (move-to-front and evict-from-back both $O(1)$).

Patterns

Interviewer says	Think
"design LRU cache"	DLL (recency order) + hashmap(key→node)
"undo/redo, browser history"	DLL with a movable "current" pointer
" $O(1)$ delete given a node reference"	Doubly linked list

Complexity Table

Operation	Time	Space	Notes
Insert (given node)	$O(1)$	$O(1)$	
Delete (given node)	$O(1)$	$O(1)$	advantage over SLL
Search	$O(n)$	$O(1)$	
Reverse traverse	$O(n)$	$O(1)$	impossible in SLL

Interview Follow-ups — Answered

- **"Implement LRU cache with $O(1)$ get/put"** — DLL in recency order (front = most recent) + hashmap key→node*. **get**: hashmap lookup, then unlink and re-insert at front, $O(1)$. **put**: if key exists, update + move to front; else insert at front and, if over capacity, evict the tail node and erase its hashmap entry. Use two dummy sentinels (head/tail) to avoid null checks.
- **"How would you make this thread-safe?"** Guard the DLL + hashmap with a mutex around each operation, or use a lock-free DLL (Harris's marked pointers, hazard pointers) if lock-free is required — trades simplicity for throughput under contention.
- **"Memory overhead vs. SLL/array?"** DLL node = payload + 2 pointers (vs. 1 for SLL, 0 extra for array); arrays win on cache locality and memory but lose $O(1)$ arbitrary insert/delete.

Tricks

- Combine with a hashmap (key → node*) whenever " $O(1)$ delete by key" is needed (LRU, LFU).
- Two sentinels (dummy head *and* dummy tail) give every real node a non-null **prev/next**, eliminating null checks entirely.

Common Bugs

- Updating **next** but forgetting the matching **prev** (or vice versa) — the classic DLL bug.
- Forgetting to update head/tail sentinels after removing the first/last real node.

Checklist

1. Updated *both* directions of every affected edge?
2. Do I need sentinel head/tail nodes?
3. Pairing this with a hashmap for $O(1)$ key-based access?

Practice Problems

Difficulty	Problem	Source
Medium	LRU Cache	LeetCode 146
Medium	Design Browser History	LeetCode 1472
Medium	Flatten a Multilevel Doubly Linked List	LeetCode 430
Hard	LFU Cache	LeetCode 460
Hard	All O`one Data Structure	LeetCode 432

3 Circular Linked List

One-line definition

A linked list (singly or doubly) where the last node's **next** points back to the first node instead of NULL, forming a ring with no natural end.

Intuition

```

+--> [10] --> [20] --> [30] --+
|                                     |
+-----+

```

No NULL terminator — every traversal needs an explicit stopping condition ("until back to start").

Graph Intuition

A directed cycle graph. Terminate with **cur != head** (checked after the first step) or a counted loop, never **cur != nullptr** — that mismatch is the top cause of infinite loops in circular-list code.

Operations

1. Traversal

```

void traverse(Node* head) {
    if (!head) return;
    Node* cur = head;
    do { process(cur); cur = cur->next; } while (cur != head);
}

```

2. Insertion / Deletion

Same as singly/doubly operations, plus: re-point the last node's **next** back to **head** whenever head changes, and handle the single-node self-loop (**node->next == node**).

3. Detecting circularity

Exactly Floyd's cycle detection — a circular list is "a list with a cycle that includes every node."

Patterns

Interviewer says	Think
"elimination game / every kth person"	circular list or Josephus recurrence
"round robin scheduler"	circular list with a "current" pointer
"detect if a list is circular"	Floyd's algorithm

Interview Follow-ups — Answered

- **"How do you avoid an infinite loop here?"** Never loop on `cur != nullptr`; use a do-while that checks `cur != head` after advancing, or a counted loop over the known size.
- **"How do you detect if a given list is circular vs. NULL-terminated?"** Floyd's tortoise and hare — if fast ever reaches `nullptr`, it's NULL-terminated; if `slow == fast`, it's circular.
- **"Josephus problem for large n ?"** Don't simulate with pointers ($O(nk)$); use the recurrence $J(n, k) = (J(n - 1, k) + k) \bmod n$, $J(1, k) = 0$, computed bottom-up in $O(n)$.

Tricks

- Round-robin problems: a circular list avoids repeated wrap-around index arithmetic.
- Small n : simulate directly. Large n : use the Josephus recurrence above.

Common Bugs

- `while (cur != nullptr)` instead of do-while/counted loop $\Rightarrow$ infinite loop.
- Forgetting the single-node self-loop case during deletion (must null `head`, not leave a dangling self-reference).

Checklist

1. Termination is "back to start," not "hit null."
2. 0-node and 1-node (self-loop) handled explicitly.
3. Deleting head: find the last node first to repoint its `next`.

Practice Problems

Difficulty	Problem	Source
Easy	Design Circular Queue	LeetCode 622
Medium	Split a Circular Linked List	Various judges
Medium	Josephus Problem	CSES / Codeforces
Hard	Design Circular Deque	LeetCode 641

4 Sentinel / Dummy / Header Nodes

One-line definition

A permanently-allocated placeholder node (no "real" data) at the head (and/or tail) of a list, used purely to eliminate special-casing in insert/delete logic.

Intuition

Without dummy:

```
head -> [A] -> [B] -> NULL
(deleting A needs special
 case: "update head")
```

With dummy:

```
dummy -> [A] -> [B] -> NULL
head = dummy.next
(deleting A is now the SAME
 code path as deleting B)
```

A dummy node converts "operations at the boundary" into "operations in the middle," so one code path handles every case.

Why it works

Every "is this the head?" bug disappears because `dummy.next` *is* the head and `dummy` itself is never removed — $O(1)$ extra memory pays for itself in correctness.

Usage Patterns

- **Merge two sorted lists:** dummy anchors the result; return `dummy.next`.
- **Delete matching a condition:** `prev` starts at `&dummy`, so deleting the very first node needs no special branch.
- **Remove Nth from end:** deleting the head is identical code to any other deletion.

```
// Canonical dummy-node skeleton
Node dummy(0); dummy.next = head;
Node *prev = &dummy, *cur = head;
while (cur) {
    if (shouldDelete(cur)) { prev->next = cur->next; /* delete cur; */ }
    else { prev = cur; }
    cur = prev->next == cur ? cur->next : prev->next;
}
return dummy.next;
```

Header Node vs Dummy Node vs Tail Pointer

Concept	Purpose	Typical use
Dummy/Sentinel node	Simplify boundary logic	Insert/delete-heavy algorithms
Header node	Store metadata (size, etc.) at position 0	Some textbook list ADTs
Tail pointer	$O(1)$ append	Queues, LRU list, append-heavy lists

Interview Follow-ups — Answered

- **"Why not just special-case `head == nullptr`?"** You can, but every extra branch is an extra place to introduce a bug; a dummy collapses N special cases into 0, and interviewers read this as a signal of experience.
- **"Does the dummy count toward the list's length/output?"** No — it's scaffolding only; never include it when computing size or building the output list.
- **"When would you use two sentinels instead of one?"** Doubly linked lists that need $O(1)$ insert/delete at *both* ends without null checks (e.g., LRU cache) — one dummy head, one dummy tail.

Tricks

- Ask: "would a dummy remove an `if (head == ...)` branch here?" If yes, use one.
- DLL: use *two* sentinels (dummy head *and* dummy tail) so every real node always has a non-null `prev/next` — exactly how LRU cache implementations avoid null checks entirely.

Common Bugs

- Returning `head` instead of `dummy.next` (stale reference if the original head was deleted).
- Accidentally including the dummy when computing length or building output.

Checklist

1. Special-casing "is this the head/tail"? Consider a dummy.
2. Returned `dummy.next`, not the original `head` variable?
3. DLL: sentinels needed on both ends?